

Toutes les définitions possibles sur React :

-React.js est une bibliothèque JavaScript open-source développée par Facebook qui permet de créer des interfaces utilisateur interactives et réutilisables.

-Un composant React est une entité réutilisable qui encapsule la logique.

-JSX est une extension de syntaxe utilisée dans React pour décrire la structure et l'apparence des composants.

-Babel est un transpileur (ou compilateur source-à-source) JavaScript qui convertit du code JavaScript moderne (ES6+) en une version compatible avec les navigateurs et environnements plus anciens.

-Les propriétés (props) sont des objets JavaScript utilisés pour passer des données d'un composant parent à un composant enfant .

-L'état (state) est un objet JavaScript qui représente les données internes d'un composant.

-Cycle de vie d'un composant : Série d'étapes (montage, mise à jour, démontage) subies par un composant classe. Dans les composants fonctions, on utilise le Hook `useEffect` pour reproduire ce comportement.

-Hooks : Fonctions spéciales qui permettent d'utiliser l'état et d'autres fonctionnalités de React dans des composants fonctions (sans écrire une classe).

-React Router : Bibliothèque standard pour la navigation et le routage dans une application React (création de SPA - Single Page Application).

-API : Ensemble de règles qui permet à deux logiciels de communiquer. Une API REST expose des endpoints (URL) pour effectuer des opérations CRUD (GET, POST, PUT, DELETE) sur des données.

-Axios : Bibliothèque cliente HTTP basée sur Promises, utilisée pour envoyer des requêtes **AJAX** à une API (plus simple et puissante que `fetch`). Ex: `axios.get(url)`.

-Redux : Bibliothèque de gestion d'état prévisible pour les applications JavaScript. Il maintient l'état de l'application dans un store global unique, accessible par tous les composants.

-Store : Objet global qui contient tout l'état de l'application. On ne le modifie pas directement.

-Actions : Objets JavaScript qui décrivent ce qui s'est passé dans l'app. Elles doivent avoir une propriété `type`.

-Reducers : Fonctions pures qui prennent l'état précédent et une action, et retournent le nouvel état. C'est le seul endroit où on calcule le nouvel état.

-dispatch : Méthode du store pour envoyer une action afin de mettre à jour l'état.

-Redux Toolkit : Package officiel et recommandé pour écrire du Redux moderne. Il simplifie la configuration, réduit le code "boilerplate" et inclut des outils puissants.

-configureStore() : Fonction du `redux-toolkit` qui simplifie la création du store (inclut automatiquement le middleware `Redux Thunk` et active les `DevTools`).

-SPA (Single Page Application) : React est conçu pour ça. Une seule page HTML, le contenu est mis à jour dynamiquement via JavaScript sans rechargement complet.

-MPA (Multi Page Application) : Chaque action charge une nouvelle page HTML depuis le serveur (approche traditionnelle).

-Microservices : L'application est découpée en petits services indépendants qui communiquent via API. Avantage : Évolutivité, déploiement indépendant.

-Virtual DOM (DOM Virtuel) : Concept clé de React. Représentation légère en mémoire de l'UI. React compare cette copie avec le DOM réel et ne met à jour que les éléments qui ont changé (optimisation des performances).

-Un middleware dans React.js est une fonction qui s'interpose entre l'envoi d'une action et son traitement par les reducers.

Les commandes d'installation :

1-Créer une nouvelle application React (méthode officielle)

```
npx create-react-app nom-du-projet
```

2-Se déplacer dans le dossier

```
cd nom-du-projet
```

3-Démarrer le serveur de développement

```
npm start
```

4-React Router (navigation)

```
npm install react-router-dom
```

5-Axios (requêtes HTTP)

```
npm install axios
```

6-Redux (gestion d'état)

```
npm install redux react-redux
```

7-Redux Toolkit

```
npm install @reduxjs/toolkit react-redux
```

7-Pour démarrer JSON server

```
npx json-server -w data/db.json -p 8000
```

Cycle de vie D'un composant (Classe/Fonction) :

1- Classe :

1. Mounting (Montage) : Le composant est créé et inséré dans le DOM.

- **constructor()** : Initialise l'état, lie les méthodes.
- **render()** : Obligatoire - Retourne le JSX.
- **componentDidMount()** : Exécuté après le premier rendu. Idéal pour les appels API, les abonnements.

2. Updating (Mise à jour) : Le composant est mis à jour quand state ou props changent.

- **shouldComponentUpdate()** : Permet d'optimiser en évitant des rendus inutiles (retourne true/false).
- **render()** : Re-rendu.
- **componentDidUpdate(prevProps, prevState)** : Exécuté après la mise à jour. Pour comparer anciennes/nouvelles valeurs.

3. Unmounting (Démontage) :

- **componentWillUnmount()** : Nettoyage (annuler timers, abonnements)

1- Fonction:

useEffect - Le remplacement universel :

```
useEffect(() => {  
  // Code = componentDidMount + componentDidUpdate  
  return () => {  
    // Cleanup = componentWillUnmount  
  };  
}, [dependencies]); // Tableau de dépendances
```

- **[] vide** = Exécute une seule fois (componentDidMount).
- **[state]** = Exécute quand state change (componentDidUpdate pour ce state).
- **Pas de tableau** = Exécute à chaque rendu (dangereux !).